

DefFusionNet: Learning Multimodal Goal Shapes for Deformable Object Manipulation via a Diffusion-based Probabilistic Model

Bao Thach¹, Siyeon Kim¹, Britton Jordan¹, Mohanraj Shanthi¹, Tanner Watts¹,
Shing-Hei Ho¹, James M. Ferguson¹, Tucker Hermans^{1,2}, Alan Kuntz¹

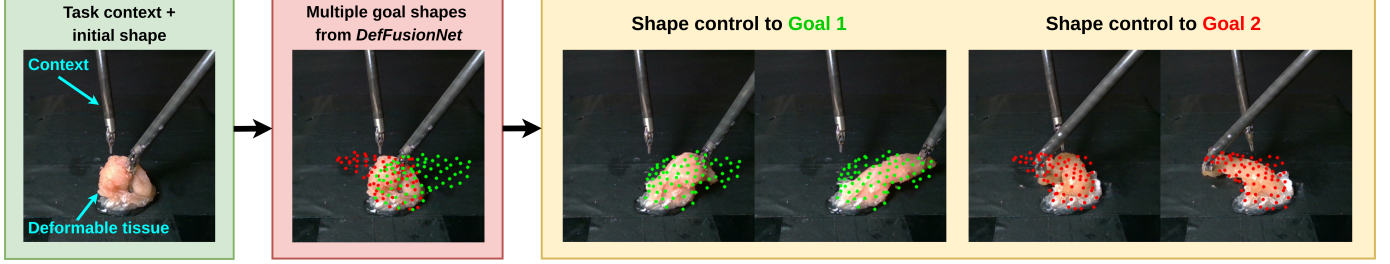


Fig. 1. **Multimodal Contextual Shape Servoing:** Given the task context and initial shape of the deformable object, *DefFusionNet* predicts multiple valid goal point clouds (red and green points) by learning a diffusion-based probabilistic model from human demonstrations. Conditioned on one of these predicted goals, *DeformerNet* then computes actions to manipulate the object into the desired shape and successfully accomplish the surgical retraction task, ensuring the tissue is pulled aside without colliding with the surgical tool.

Abstract—Deformable object manipulation is critical to many real-world robotic applications, ranging from surgical robotics and soft material handling in manufacturing to household tasks like laundry folding. At the core of this important robotic field is shape servoing, a task focused on controlling deformable objects into desired shapes. The shape servoing formulation requires the specification of a goal shape. However, most prior works in shape servoing rely on impractical goal shape acquisition methods, such as laborious domain-knowledge engineering or manual manipulation. *DefGoalNet* previously posed the current state-of-the-art solution to this problem, which learns deformable object goal shapes directly from a small number of human demonstrations. However, it significantly struggles in multi-modal settings, where multiple distinct goal shapes can all lead to successful task completion. As a deterministic model, *DefGoalNet* collapses these possibilities into a single averaged solution, often resulting in an unusable goal. In this paper, we address this problem by developing *DefFusionNet*, a novel neural network that leverages the diffusion probabilistic model to learn a distribution over all valid goal shapes rather than predicting a single deterministic outcome. This enables the generation of diverse goal shapes and avoids the averaging artifacts. We demonstrate our method’s effectiveness on robotic tasks inspired by both manufacturing and surgical applications, both in simulation and on a physical robot. Our work is the first generative model capable of producing a diverse, multi-modal set of deformable object goals for real-world robotic applications. We publish all code and data at: <https://sites.google.com/view/deffusionnet/home>.

I. INTRODUCTION

Deformable object manipulation is central to many robotic tasks that have the potential to significantly improve human lives, widely ranging from healthcare (e.g., delicate tissue manipulation in robotic surgery), to manufacturing (e.g., handling materials such as textiles, bags, and boxes), to everyday household tasks (e.g., folding laundry, preparing food).

While rigid-body manipulation has been extensively studied [1], autonomous 3D deformable object manipulation remains under-explored [2, 3], largely due to its inherent challenges [2, 4]. Deformable objects require infinitely many degrees of freedom (DOF) to fully describe their shapes, unlike the 6 DOF required to represent rigid objects. Their complex dynamics [5] also make model derivation for manipulation costly. Our prior work, *DeformerNet* [6], was the first to address both issues by using point clouds as state representations and learning low-dimensional latent features via a neural network. *DeformerNet* addresses shape servoing, a core deformable manipulation task focused on controlling objects toward desired shapes. It takes current and goal point clouds as input and outputs the robot action needed to reach the target shape.

However, a key limitation of *DeformerNet* and other shape servoing approaches lies in their reliance on explicitly defined geometric goal shapes, such as a goal point cloud. In prior work, these goals were acquired through manual demonstrations or extensive domain-specific engineering—processes that are time-consuming and ill-suited for practical robotic deployment. This reliance on an inaccessible goal parameterization remains a key barrier to the practical adoption of shape servoing algorithms.

Our recent work, *DefGoalNet* [7], takes an important step toward overcoming this limitation. It introduces a neural network that learns to infer goal shapes for deformable objects

This material is supported in part by the Advanced Research Projects Agency for Health (ARPA-H) under grant number D24AC00415, and by the NSF Foundational Research in Robotics (FRR) Faculty Early Career Development Program (CAREER) under grant number 2144348. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of ARPA-H and NSF.

¹Robotics Center and Kahlert School of Computing, University of Utah, Salt Lake City, UT 84112, USA. ²NVIDIA Corporation, Seattle, WA, USA. {bao.thach, alan.kuntz}@utah.edu

directly from human demonstrations. *DefGoalNet* overcomes the need for manually defined goal shapes by predicting goal point clouds directly from task-specific sensory input, enabling seamless integration with *DeformerNet* for shape servoing.

Despite its success across a variety of robotic tasks [7], *DefGoalNet* exhibits a significant limitation: it struggles to learn multi-modal goal shape distributions. Here, multi-modal refers to scenarios where, given a fixed current shape and environmental context, multiple distinct goal shapes are equally effective in accomplishing the task. For example, a robot manipulating a deformable object might twist it either to the left or to the right—both resulting in valid outcomes (Fig. 1). In such cases, deterministic models like *DefGoalNet* “average” the left and right goal shapes (Fig. 2), producing an impractical, ambiguous result that often causes task failure. Crucially, multi-modality is not rare but fundamental in real-world tasks involving deformable objects. For instance, folding a towel in halves, thirds, or quarters are all equally valid ways to achieve compactness, highlighting the importance of modeling diverse goal states. Ignoring this property might risk failing a wide range of robotic tasks.

Addressing this critical limitation, in this paper, we introduce a novel neural network *DefFusionNet*, a major advancement over *DefGoalNet* that models a distribution over all possible goal shapes rather than predicting a single deterministic outcome. *DefFusionNet* leverages a diffusion architecture [8], a probabilistic model theoretically capable of capturing any distribution, regardless of its complexity, dimensionality, or modality. This enables the generation of multiple diverse goal shapes conditioned on the current object shape and context.

As part of our novel learning-from-demonstration shape servoing pipeline, we first train *DefFusionNet* on a set of multi-modal demonstration trajectories collected from diverse demonstration policies (e.g., different human demonstrators performing the same task in distinct ways). This enables the model to learn a distribution over all possible goal shapes that accomplish the task given the current context. At runtime, *DefFusionNet* takes as inputs the current deformable object point cloud and the contextual point cloud, and samples a goal shape from the learned goal distribution corresponding to a successful outcome under that context. This predicted goal shape is then passed to *DeformerNet*, which computes the corresponding end-effector action to deform the object toward the goal shape, executed in a closed-loop fashion (see Fig. 1). Successfully deforming the object to match the predicted goal shape from *DefFusionNet* is considered equivalent to task success.

The combination of *DefFusionNet* and *DeformerNet* enables a reformulation of the robot policy, requiring *only* inputs that are practically available: the current point cloud of the deformable object and a second point cloud encoding task-specific context.

We evaluate our approach on both surgery-inspired and manufacturing-inspired robotic tasks. In the surgical domain, we perform experiments on tissue retraction, a common procedure where tissues are pulled to one side to generate sufficient tension between the tissue and the underlying anatomy. In the manufacturing domain, we evaluate performance on an object

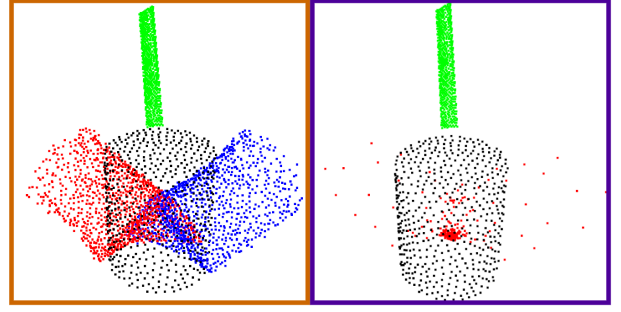


Fig. 2. Simulated retraction results comparing predicted goals from *DefFusionNet* (left) to *DefGoalNet* (right). *DefFusionNet* effectively captures the underlying bimodal goal distribution, producing two distinct goal shapes (red and blue). In contrast, *DefGoalNet* averages the two modes, resulting in a single unrealistic and physically infeasible goal point cloud (red).

packaging task. Experiments are conducted both in simulation and on physical robot hardware. Across all tasks, the multi-modal goal shapes predicted by *DefFusionNet* consistently lead to high success rates, outperforming the current state-of-the-art goal generation method *DefGoalNet*.

We are the first work to introduce a generative model capable of producing a diverse, multi-modal set of deformable object goals. By offering a more expressive and flexible modeling framework, *DefFusionNet* brings deformable object manipulation closer to practical real-world applications.

II. RELATED WORK

Recent advances in machine learning have empowered robots to manipulate rigid objects by effectively leveraging rich, high-dimensional sensory inputs such as 3D point clouds [9–14]. Neural networks, in particular, have become central to tackling complex robotic perception and control problems, including shape reconstruction [14], object pose estimation [13], and grasp synthesis [9, 10, 13, 14]. Furthermore, sophisticated learning frameworks have enabled robots to execute long-horizon, multi-step tasks—such as clearing a table of assorted food items—by integrating a diverse set of learned behaviors [15]. Building on these successes, we explore a learning-based framework aimed at controlling the 3D shape of deformable objects.

Traditionally, shape servoing has been addressed through predominantly model-based or learning-free techniques [16–21]. These methods typically rely on manually selected feature points to represent the deformable object, making them susceptible to sensor noise and limiting their ability to generalize to novel object geometries. Shetab-Bushehri *et al.* [21] propose a 3D lattice-based representation that enables accurate 3D control of deformable shapes. However, their method relies on the assumption of consistent feature correspondences over time, which is difficult to uphold in real-world robotic deployments.

Learning-based approaches have emerged as a promising direction for 3D shape control by leveraging the generalization strength of modern neural networks. Hu *et al.* [22] utilize Fast Point Feature Histograms (FPFH) [23] to represent the state of deformable objects within a learning framework. However, Thach *et al.* [24] demonstrate that this representation, along with the associated network architecture, struggles to model the intricate dynamics inherent to 3D deformable objects. A

more recent advancement, *DeformerNet* [6, 24], represents the current state-of-the-art in shape servoing. It employs a PointConv-based [25] neural network that takes as input the current and goal point clouds and predicts the corresponding robot action to deform the object toward the goal shape. However, a key limitation of *DeformerNet* and other shape servoing approaches is their reliance on explicitly defined goal shapes.

Although point cloud generative models have achieved impressive results in computer vision and graphics [26–29], their integration into robotic applications has so far been relatively underexplored.

In the context of goal generation for robotic tasks, recent works [30, 31] have proposed generative models capable of synthesizing goal images from natural language instructions. However, these approaches are not well-suited for reasoning about the complex geometries of deformable objects, and 2D image goals are insufficient for our 3D shape servoing task. Instead of relying on visual goal representations (e.g., images, videos, or point clouds), some recent works leverage advances in large language models to use text-based task descriptions directly as the goal [32, 33]. In these approaches, the robot interprets the textual goal and maps it directly to actions in an end-to-end fashion, eliminating the need for intermediate goal representations. However, we argue that textual instructions alone are insufficient to fully specify the complexity of deformable shape control tasks.

In the context of learning robotic policies from multi-modal demonstration datasets, Diffusion Policy [34] represents the current state-of-the-art. It leverages a diffusion probabilistic model to map image observations directly to a distribution over robot actions, achieving superior performance compared to deterministic models. Stein Variational Belief Propagation (SVBP) [35] achieves a similar objective using a different probabilistic modeling approach based on Stein Variational Inference. Our work draws inspiration from these approaches but diverges by predicting goal representations instead of robot actions directly, delegating the action prediction to *DeformerNet*. We argue that this separation leads to significantly better sample efficiency. Training Diffusion Policy or SVBP requires expensive data collection processes, involving human supervision and carefully designed demonstration scripts. In contrast, data for training *DeformerNet* is substantially cheaper, as it can be collected at scale through simulation or random policies. Data for training the goal generation component is still expensive, but we need far less data for it [7].

III. PROBLEM FORMULATION

We consider the problem of bimanually manipulating a 3D deformable object to accomplish a designated task T . The object is defined as $\mathcal{O} \subset \mathbb{R}^3$, and evolves dynamically throughout the manipulation process. Since it is typically infeasible to observe the entire geometry of $\mathcal{O}$ directly, we operate on a partial-view point cloud $\mathcal{P} \subset \mathcal{O}$, which captures only a portion of the object’s surface. The current point cloud observation of the deformable object is denoted as $\mathcal{P}_c$.

The robot interacts with the deformable object by grasping it at two manipulation points, denoted $\{\mathbf{p}_{m_i}\}_{i=1:2}$, and

manipulating it through corresponding end-effector motions. We formulate each robot action $\mathcal{A}$ as a pair of rigid transformations in $\mathcal{SE}(3) \times \mathcal{SE}(3)$, representing the changes in the poses of the two robot end-effectors. For tasks that involve only single-arm manipulation, this formulation simplifies to a single manipulation point and a single transformation matrix.

We cast the task as a form of contextual learning problem, where the robot must infer how to act based on two sources of information available at the start of execution: the current shape of the deformable object and the observable context from the surrounding environment. Together, these observations define a distribution over ways that the task can be successfully executed. To encode the surrounding context, we define a task-specific contextual point cloud $\mathcal{P}_T$, which captures relevant environmental structure rather than the geometry of the deformable object itself. This contextual representation may reflect nearby objects, environmental landmarks, or other critical features that define task success. The specific design of $\mathcal{P}_T$ depends on the task and will be detailed in Sec. V. For instance, in a surgical setting where a robot must elevate a tissue layer from a kidney, $\mathcal{P}_T$ might include a partial view of the kidney that is visible at the beginning of the procedure.

Given a task T , we assume the availability of a dataset $\mathcal{D}$ containing expert demonstrations that illustrate how to successfully complete the task across a variety of contexts. Each demonstration trajectory τ is a time-ordered sequence of M point cloud observations, $(\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_M)$, where each $\mathcal{P}_i$ captures the deformable object’s state at a specific timestep during the demonstration. Our objective is to train a policy that enables the robot to perform task T autonomously in novel contexts not seen during training. The robot learning process is supervised with the expert demonstrations, each annotated with the corresponding task context and object point cloud observations.

In contrast to [7], our dataset $\mathcal{D}$ is explicitly multi-modal: the demonstration trajectories are collected from diverse demonstration policies (e.g., different human demonstrators performing the same task in distinct ways), all of which are equally effective and lead to successful task completion. Note that multi-modality is not an unrealistically artificial constraint we impose in this work; rather, it reflects the reality of most large-scale robotic datasets currently [36], which include demonstrations from different institutions, robot designs, scripted policies, etc.

IV. METHOD

Our framework assumes that a set of N goal shapes exists $\mathcal{G} = \{\mathcal{O}_{g_i}\}_{i=1:N}$, such that if the object reaches *any* goal configuration $\mathcal{O}_{g_i}$, the task succeeds. Building on this insight, we break the initial complex problem into two distinct sub-problems. The first focuses on contextual goal learning: given the current object state and task context, the objective is to model a distribution over all possible goal point clouds $\mathcal{P}_g$ that would lead to successful task outcomes, formalized as $p_\theta(\mathcal{P}_g \mid \mathcal{P}_c, \mathcal{P}_T)$. Here, θ denotes the parameters of a neural network, described later in the following section. This goal generation model is trained on a demonstration dataset $\mathcal{D}$

associated with task T . At runtime, *DefFusionNet* conditions on the current object point cloud and context, and samples a goal point cloud from the learned distribution that is expected to lead to a successful task execution under that context: $\mathcal{P}_g \sim p_\theta(\mathcal{P}_g | \mathcal{P}_c, \mathcal{P}_T)$.

Unlike the current state-of-the-art *DefGoalNet*, which uses a standard deterministic neural network, our *DefFusionNet* represents the goal generator as a probabilistic model. This formulation enables the learning of multiple valid goal shapes that can all successfully complete the same task. Specifically, we adopt techniques from *Diffusion Probabilistic Models* [8], a state-of-the-art class of algorithm previously used to model distributions of images [8], videos [37], and music [38], to learn our conditional goal distribution $p_\theta(\mathcal{P}_g | \mathcal{P}_c, \mathcal{P}_T)$.

With the goal point cloud generated, we turn our attention to the second sub-problem of goal-conditioned shape control: how to deform the object toward the goal shape. To this end, we learn a policy that takes as input the current point cloud, the goal point cloud, and a selected manipulation point, and outputs a corresponding robot action: $\pi(\mathcal{P}_c, \mathcal{P}_g, \mathbf{p}_m) = \mathcal{A}$. By executing this policy iteratively, the robot incrementally reshapes the object, steering it closer to the target shape until the task is completed. Manipulation points are determined using the *dense predictor* network introduced in [6].

As *DeformerNet* [6] already offers a robust solution for the second sub-problem, our focus in the remainder of this section will solely be on the problem of contextual goal learning. We begin by outlining the theoretical foundations of diffusion probabilistic models, which form the basis of our goal generation approach. We then introduce the architecture of our *DefFusionNet* network. Following this, we describe the training algorithm. Afterward, we outline the inference-time sampling procedure used to produce diverse goal shapes from the learned conditional distribution $p_\theta(\mathcal{P}_g | \mathcal{P}_c, \mathcal{P}_T)$. To complete the pipeline, we demonstrate how the generated goal from *DefFusionNet* serve as an input to the goal-conditioned shape controller in *DeformerNet*, allowing the robot to successfully perform the task T .

A. Diffusion Probabilistic Model Overview

To simplify the mathematical notation for diffusion, only in this section, we use $\mathbf{X} = \{\mathbf{x}_i\}_{i=1}^N$ to represent the goal point cloud $\mathcal{P}_g$ consisting of N points. Additionally, we use $\mathbf{c}$ to represent all conditioning observations, which include the initial point cloud $\mathcal{P}_c$ and the contextual point cloud $\mathcal{P}_T$. Our objective is to learn the conditional goal distribution:

$$p_\theta(\mathcal{P}_g | \mathcal{P}_c, \mathcal{P}_T) = p_\theta(\mathbf{X} | \mathbf{c}) = \prod_{i=1}^N p_\theta(\mathbf{x}_i | \mathbf{c}) \quad (1)$$

through the diffusion process.

Following the general diffusion framework in [8, 39], we train a *noise predictor* neural network to implicitly model any arbitrarily complex distribution, given samples from the distribution, using the *forward* diffusion process. At inference, we generate samples from the learned distribution via the *reverse* diffusion process.

In the **forward diffusion process**, we incrementally add a small amount of noise to the goal point cloud $\mathbf{X}$ over T timesteps. By the end of these T steps, $\mathbf{X}$ transforms from a meaningful point cloud to a fully noisy distribution. Following the convention of [8, 39], we define the diffusion kernel as:

$$\mathbf{x}_i^{(t)} = \sqrt{\alpha_t} \mathbf{x}_i^{(0)} + \sqrt{1 - \alpha_t} \epsilon \quad (2)$$

where $\mathbf{x}_i^{(0)}$ is the original ‘‘clean’’ point, and $\mathbf{x}_i^{(t)}$ is the point after adding noise t times. α_t is a scalar hyperparameter that directly controls the amount of noise added to the original point cloud during each step of the forward diffusion process. The cumulative product $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ represents the proportion of the original signal retained after t diffusion steps.

Our goal is to generate goal point clouds that accomplish successful task outcome. We treat this generation process as the **reverse diffusion process**, where the objective is to predict the noise that was added to the original point cloud using a neural network. By subtracting the predicted noise from the noisy point cloud, we recover the desired goal shape. Unlike the forward process which simply adds noise to the points, reverse diffusion requires learning a neural network $\mu_\theta(\mathbf{x}_i^{(t)}, t, \mathbf{c}, \mathbf{z})$ from data to predict the noise (Sec. IV-C). This network is conditioned on the current noisy point cloud $\mathbf{x}_i^{(t)}$, conditioning observations $\mathbf{c}$, time step t , and $\mathbf{z}$ —a latent encoding representing the goal point cloud. The use of $\mathbf{z}$ follows the Variational Autoencoding (VAE) framework [40], enhancing the generative capability of the model. We formulate the reverse diffusion process for generation as:

$$p_\theta(\mathbf{x}_i^{(t-1)} | \mathbf{x}_i^{(t)}, \mathbf{c}, \mathbf{z}) = \mathcal{N}(\mathbf{x}_i^{(t-1)} | \mu_\theta(\mathbf{x}_i^{(t)}, t, \mathbf{c}, \mathbf{z}), \beta_t I), \quad (3)$$

where μ_θ , hereafter referred to as the *noise predictor* network parameterized by θ , is the heart of the diffusion process.

We can then compute our objective distribution as follows:

$$\begin{aligned} p_\theta(\mathbf{x}_i | \mathbf{c}) &= p_\theta(\mathbf{x}_i^{(0:T)} | \mathbf{c}, \mathbf{z}) \\ &= p(\mathbf{x}_i^{(T)}) \prod_{t=1}^T p_\theta(\mathbf{x}_i^{(t-1)} | \mathbf{x}_i^{(t)}, \mathbf{c}, \mathbf{z}) \end{aligned} \quad (4)$$

where $\mathbf{x}_i^{(T)} \sim \mathcal{N}(0, I)$ is random point sampled from a unit Gaussian.

B. DefFusionNet Architecture

Figure 3 provides a comprehensive overview of the *DefFusionNet* architecture. Following the VAE framework [40], we first learn the latent representation of the goal point cloud distribution. The original goal point cloud $\mathcal{P}_g$ is passed through an encoder network g_g . We adopt PointNet [41] for the encoder architecture. The encoder outputs two components: a 256-dimensional mean vector μ and a 256-dimensional standard deviation vector σ , which parameterize a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$. Using the reparameterization trick, a latent vector $\mathbf{z}$ is sampled from this Gaussian distribution as $\mathbf{z} = \mu + \sigma \odot \xi$, where $\xi \sim \mathcal{N}(0, I)$ and $(\mu, \sigma) = g_g(\mathcal{P}_g)$. This latent vector serves as a compact representation of the goal point cloud,

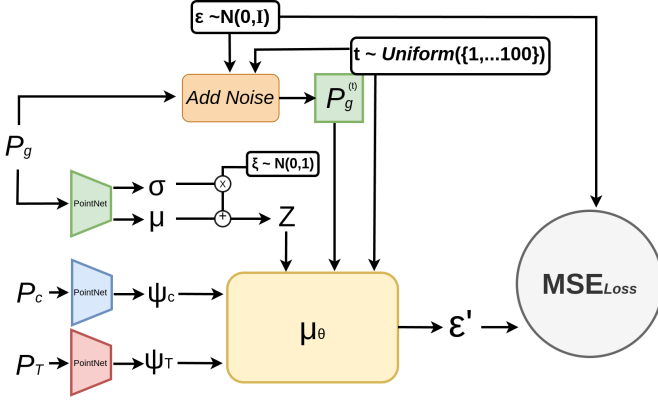


Fig. 3. *DefFusionNet* architecture for learning the multimodal goal distribution for a deformable object manipulation task.

capturing its structural and spatial properties. Overall, we denote this sampling process from the VAE as $\mathbf{z} \sim q_\varphi(\mathbf{z} | \mathcal{P}_g)$, where q_φ is the encoder-defined posterior.

We also feed the current point cloud $\mathcal{P}_c$ and contextual point cloud $\mathcal{P}_T$ into two identical PointNet [41] encoder channels g , generating two 256-dimensional vectors: the current feature vector $\psi_c = g(\mathcal{P}_c)$ and the contextual feature vector $\psi_T = g(\mathcal{P}_T)$.

Next, we synthesize a noisy point cloud $\mathcal{P}_g^{(t)}$ by uniformly sampling a time step $t \sim \mathcal{U}\{1, \dots, 100\}$, then sampling a **target noise vector** $\epsilon \sim \mathcal{N}(0, I)$ of shape $N \times 3$ (N is the number of points in the goal point cloud), and finally adding noise to the goal point cloud using Eq. 2. The noise predictor network μ_θ takes as inputs this noisy state $\mathcal{P}_g^{(t)}$, two feature vectors ψ_c and ψ_T , time step t , and latent shape representation $\mathbf{z}$. It outputs the **predicted noise vector** ϵ' , which can be compared with the original target ϵ to create the training signal. The composite *DefFusionNet* goal generator network thus takes the form:

$$\epsilon' = \mu_\theta(t, \mathcal{P}_g^{(t)}, \mathbf{z}, \psi_c, \psi_T) = \mu_\theta(t, \mathcal{P}_g^{(t)}, \mathbf{z}, g(\mathcal{P}_c), g(\mathcal{P}_T)) \quad (5)$$

where $t \sim \mathcal{U}\{1, \dots, 100\}$ and $\mathbf{z} \sim q_\varphi(\mathbf{z} | \mathcal{P}_g)$. Details about the architecture of the *noise predictor* are provided in Sec. IV-C.

C. Noise Predictor Architecture

Figure 4 provides a comprehensive overview of the *noise predictor* architecture. We first transform scalar t to a 3D time-embedding vector using its corresponding α_t value. Next, we process and combine all features from $\mathbf{z}$, ψ_c , and ψ_T using a series of concatenation operators and fully-connected layers, which eventually produces a 512-dimension feature vector. We concatenate this vector with the time-embedding vector to produce the final 515-dimension context embedding vector $\phi = f(t, \mathbf{z}, \psi_c, \psi_T)$ encompassing all features except for the noisy goal point cloud $\mathcal{P}_g^{(t)}$.

We need an effective way to incorporate ϕ into the last input $\mathcal{P}_g^{(t)}$. Following the concepts in Long Short Term Memory (LSTM) [42], we send ϕ through two parallel fully-connected layers to produce a 3D gate vector ϕ_g and a 3D bias vector

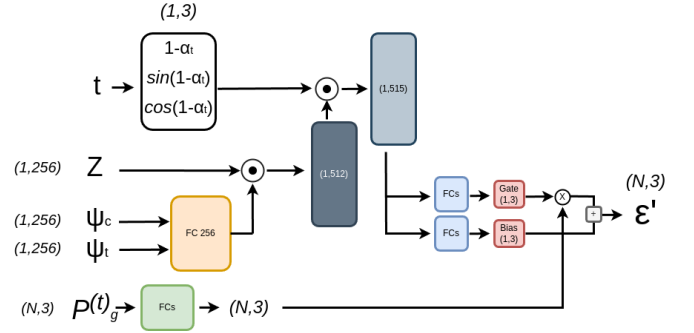


Fig. 4. *Noise Predictor* architecture. A submodule network within *DefFusionNet* responsible for predicting the diffusion noise vector.

ϕ_b . We can then compute the predicted noise vector as: $\epsilon' = h(\mathcal{P}_g^{(t)}) \cdot \phi_g + \phi_b$, where h is a series of fully-connected layers.

D. DefFusionNet Training Algorithm

Training *DefFusionNet* (as described in Algorithm 1) follows a non-standard supervised learning approach. Instead of directly training the model to produce the goal point cloud, we train it to predict the *noise* added to the goal point cloud. As a result of this unique training approach, the inference process (explained in Sec. IV-E) relies on a completely different procedure to generate the goal point clouds by leveraging the predicted noise from the trained model.

Given a demonstration trajectory, we apply a segmentation mask over the raw point cloud observations to obtain points that belong to the object. We set the object point cloud at the beginning of the trajectory as $\mathcal{P}_c$ and the terminal object point cloud as $\mathcal{P}_g$. For the remaining points that do not belong to the object volume, we select a subset of task-relevant points and set them as the context $\mathcal{P}_T$. The design choice of what points to be included in $\mathcal{P}_T$ varies from task to task. We elaborate on our specific choices in Sec. V.

To effectively capture the complex geometries of deformable objects, we adopt a loss function that combines noise reconstruction loss and KL-divergence loss. The noise reconstruction loss leverages the standard Mean Squared Error (MSE) loss to ensure that the predicted noise from *DefFusionNet* matches with the true noise:

$$\mathcal{L}_{\text{noise-recon}} = \sum_{i=1}^N \|\epsilon_i - \epsilon'_i\|^2 \quad (6)$$

The KL divergence loss, a crucial part of any VAE training, ensures that the learned latent distribution remains close to the chosen prior distribution, typically a unit Gaussian:

$$\mathcal{L}_{\text{KL}} = \frac{1}{T} D_{\text{KL}} [q_\varphi(\mathbf{z} | \mathcal{P}_g) \| \mathcal{N}(0, I)] \quad (7)$$

It acts as a regularizer, encouraging the PointNet encoder to produce a latent space that is continuous and smooth. Since $q_\varphi(\mathbf{z} | \mathcal{P}_g)$ is designed to be a Gaussian distribution, we can easily compute this KL-divergence with a closed-form solution.

E. DefFusionNet Inference Algorithm

At inference, our objective is to sample a single goal point cloud $\mathcal{P}_g$ from the distribution of all possible goal shapes

Algorithm 1 *DefFusionNet* Training

- 1: Sample $\mathcal{P}_g, \mathcal{P}_c, \mathcal{P}_T$ from training dataset. Set $\mathcal{P}_g^{(0)} = \mathcal{P}_g$.
- 2: Sample $\mathbf{z} \sim q_\varphi(\mathbf{z} | \mathcal{P}_g^{(0)})$ from the VAE.
- 3: Sample $t \sim \mathcal{U}\{1, \dots, 100\}$.
- 4: Sample target noise: $\epsilon \sim \mathcal{N}(0, I)$.
- 5: Compute forward diffusion:

$$\mathcal{P}_g^{(t)} = \sqrt{\bar{\alpha}_t} \mathcal{P}_g^{(0)} + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

- 6: Compute predicted noise:

$$\epsilon' = \mu_\theta(t, \mathcal{P}_g^{(t)}, \mathbf{z}, g(\mathcal{P}_c), g(\mathcal{P}_T))$$

- 7: Compute reconstruction loss:

$$\mathcal{L}_{\text{noise-recon}} = \sum_{i=1}^N \|\epsilon_i - \epsilon'_i\|^2$$

- 8: Compute KL loss:

$$\mathcal{L}_{\text{KL}} = \frac{1}{T} D_{\text{KL}} \left[q_\varphi(\mathbf{z} | \mathcal{P}_g^{(0)}) \parallel \mathcal{N}(0, I) \right]$$

- 9: Compute gradient:

$$\nabla (\mathcal{L}_{\text{noise-recon}} + \mathcal{L}_{\text{KL}}); \quad \text{then perform gradient descent.}$$

that lead to task success, i.e., $\mathcal{P}_g \sim p_\theta(\mathcal{P}_g | \mathcal{P}_c, \mathcal{P}_T)$. This is done using the trained noise predictor μ_θ , the goal encoder g_g , and the current/contextual encoder g , following the reverse diffusion process.

Described in detail in Alg. 2, we begin by sampling a latent vector $\mathbf{z} \sim \mathcal{N}(0, I)$ from the goal encoder g_g , and initialize a fully-noisy goal point cloud $\mathcal{P}_g^{(100)} \sim \mathcal{N}(0, I)$. The overall idea is to iteratively subtract noise from this noisy point cloud over 100 diffusion steps, ultimately producing a “clean” goal point cloud $\mathcal{P}_g = \mathcal{P}_g^{(0)}$, which serves as the final output of the inference stage.

At each diffusion step t , the noise predictor μ_θ estimates the noise vector ϵ' based on the current noisy goal point cloud $\mathcal{P}_g^{(t)}$, the latent vector $\mathbf{z}$, and the encoded features of the current and contextual point clouds, $g(\mathcal{P}_c)$ and $g(\mathcal{P}_T)$. We then subtract the predicted noise from $\mathcal{P}_g^{(t)}$ to obtain a less noisy point cloud $\mathcal{P}_g^{(t-1)}$.

F. Integration of DefFusionNet with DeformerNet

Note that *DefFusionNet* alone cannot solve the robotic task, as it only produces the goal shape for the deformable object. Therefore, following [7], we need to pair it with *DeformerNet* [6], a neural network that takes as inputs the current and goal object point clouds along with the manipulation point position, and outputs the robot action to manipulate the deformable object toward the goal shape: $\pi_s(\mathcal{P}_c, \mathcal{P}_g, \mathbf{p}_m) = \mathcal{A}$.

DefFusionNet predicts a goal point cloud $\mathcal{P}_g \sim p_\theta(\mathcal{P}_g | \mathcal{P}_c, \mathcal{P}_T)$ based on the current object state and task context, which serves as an input to *DeformerNet*. As a result, the robot policy no longer depends on the problematic $\mathcal{P}_g$ that was difficult to obtain in previous shape servoing approaches. Instead, it is now only a function of practically accessible parameters, namely $\mathcal{P}_c$ and $\mathcal{P}_T$.

Algorithm 2 *DefFusionNet* Inference

(Sampling $\mathcal{P}_g \sim p_\theta(\mathcal{P}_g | \mathcal{P}_c, \mathcal{P}_T)$)

- 1: **Input:** Trained models: noise predictor μ_θ , goal encoder g_g , and current/contextual encoder g . Current point cloud $\mathcal{P}_c$. Contextual point cloud $\mathcal{P}_T$.
- 2: **Output:** Generated goal point cloud $\mathcal{P}_g$.
- 3: Sample $\mathbf{z} \sim \mathcal{N}(0, I)$.
- 4: Initialize fully-noisy point cloud: $\mathcal{P}_g^{(100)} \sim \mathcal{N}(0, I)$.
- 5: **for** $t = 100$ to 1 **do** Compute predicted noise:

$$\epsilon' = \mu_\theta(t, \mathcal{P}_g^{(t)}, \mathbf{z}, g(\mathcal{P}_c), g(\mathcal{P}_T)).$$

Subtract noise (reverse diffusion step):

$$\mathcal{P}_g^{(t-1)} = \frac{1}{\sqrt{\alpha_t}} \left(\mathcal{P}_g^{(t)} - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon' \right).$$

- 6: Return $\mathcal{P}_g = \mathcal{P}_g^{(0)}$.

When executed in a closed-loop manner, this policy guides the robot to successfully complete the manipulation task. Additionally, before the manipulation begins, the learned goal point cloud allows the *dense predictor* network [6] to identify appropriate manipulation points on the object for grasping. Importantly, we employ the original trained *DeformerNet* and *dense predictor* models as introduced in [6], without any additional fine-tuning. This highlights a key benefit of our framework: the decoupling of goal generation and shape control, which allows each component to be trained independently.

V. EXPERIMENTS AND RESULTS

We evaluate the performance of our method on robotic tasks inspired by both manufacturing and surgical applications, demonstrating the versatility of *DefFusionNet*. In the manufacturing domain, we consider a deformable object packaging task in which the robot must safely deform or compress an object to allow it to pass through the container opening without causing damage. For the surgical domain, we focus on the surgical retraction task, which is critical in most surgical procedures.

We examine *DefFusionNet* both in simulation and on real-world robotic hardware. In simulation, we utilize the patient-side manipulator of the da Vinci Research Kit (dVRK) surgical robot [43], using the Isaac Gym platform [44]. For physical-robot experiments, we employ the physical dVRK system for the surgical task, and two KUKA iiwa 7-DOF robotic arms equipped with Robotiq and Reflex Taktilite 2 grippers for the manufacturing task. An Intel® RealSense™ D455 camera is used to capture point clouds for perception.

A. Demonstration data collection

We now outline the data collection procedure used to train *DefFusionNet* in our experiments. First, we collect M demonstration trajectories that successfully execute the task, either through real human demonstrations or scripted robot policies. To introduce multi-modality into the demonstration dataset, we use K distinct, equally effective demonstrators, each providing a different solution to the task. From each trajectory, we extract the initial and terminal object point clouds, designating them

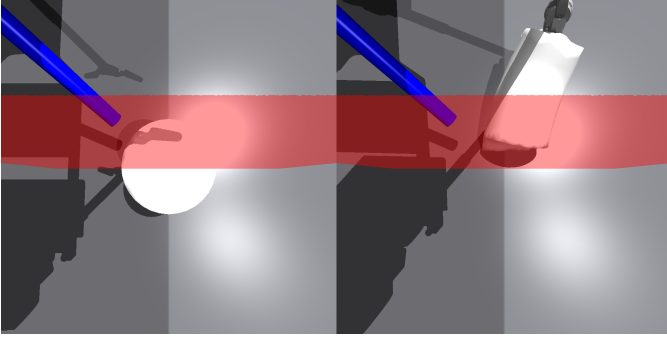


Fig. 5. Tool-conditioned surgical retraction task setup. The objective is to pull the tissue (white) to one side to create tension for subsequent cutting, while avoiding collision with the surgical tool (blue). **Left:** Initial task state, where a target plane (red) bisects the tissue into two halves. **Right:** The task is deemed successful if the tissue is fully retracted past the target plane without any collision.

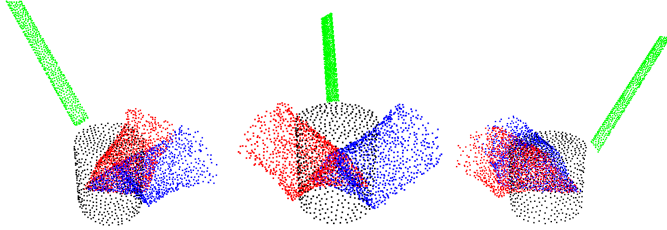


Fig. 6. Example demonstrations for the surgical retraction task. The black and green partial point clouds represent the initial tissue state and the contextual surgical tool, respectively. The red and blue point clouds depict two distinct, equally valid goal shapes collected from expert demonstrator.

as the current and goal point clouds ($\mathcal{P}_c$ and $\mathcal{P}_g$, respectively) for training. Additionally, we capture the contextual point cloud $\mathcal{P}_T$. We describe the specific setup for $\mathcal{P}_T$ and the demonstrators in the following sections, following the formal introduction of each task. Finally, we use the collected data to form training examples for *DefFusionNet*, where each input pair $(\mathcal{P}_c, \mathcal{P}_T)$ is mapped to a target output $\mathcal{P}_g$.

B. Simulation Experiments

1) *Surgery-inspired Tissue Retraction:* Here, we design a simulated robotic setup to emulate a surgical retraction task. As illustrated in Figure 5, the robot is responsible for retracting (i.e., pulling) the tissue to generate sufficient tension between the tissue and the underlying anatomy, thereby facilitating subsequent tissue cutting and removal. A critical decision during the retraction procedure is selecting a direction of retraction that avoids collisions with other surgical instruments or sensitive anatomy present in the scene. Figure 6 visualizes several feasible retraction directions based on the pose of the surgical tool. For instance, if the tool is positioned right behind the tissue, the robot should retract the tissue either to the left or to the right to avoid interference.

To train *DefFusionNet* on this task, we generate a simulated dataset consisting of tissues with diameters ranging from 1 to 7 cm, consistent with the typical distribution of tumor sizes observed in surgery, for instance in the removal of central airway obstructive (CAO) tumors [45]. The surgical tool is approximated as a cylindrical object with a diameter of 0.2 cm. For each scene, we randomize the pose of the tool to define restricted regions where tissue retraction is not allowed.

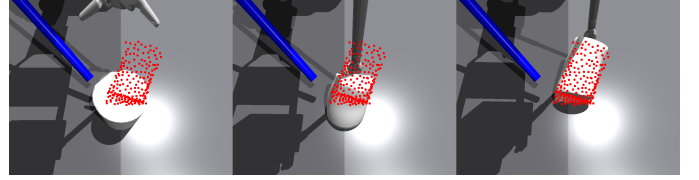


Fig. 7. Sample manipulation sequence on the surgical retraction task, in simulation. The red point cloud visualizes the goal shape generated by *DefFusionNet*.

To facilitate consistent evaluation, the tool is always placed on one side of a reference plane, hereafter referred to as the “target plane side”, which bisects the tissue into two halves along its longitudinal dimension (Figure 5). The objective is to retract the tissue toward the target plane side while avoiding collisions with the surgical tool. The contextual point cloud $\mathcal{P}_T$ for this task corresponds to the partial-view point cloud of the portion of the surgical tool that is unoccluded by the tissue.

Demonstration trajectories are generated using a scripted multimodal robot policy, as illustrated in Figure 5 and 6. To better illustrate the behavior of a multimodal model like *DefFusionNet* in contrast to a deterministic model like *DefGoalNet*, we intentionally design a controlled experiment with a two-mode goal distribution (Fig. 6). For a given tissue and tool pose, we collect two distinct goal shapes corresponding to two different retraction angles. If the surgical tool is positioned to the left of the tissue, the robot retracts the tissue to the right, selecting uniformly between two equally valid retraction angles—30° and 60° from the reference plane. A similar setup is used when the tool is on the right. If the tool is centered, the robot may retract the tissue either to the left or right at a 60° angle, with both directions considered equally effective. For each context, two demonstrations are collected: the robot grasps the tissue and retracts it along one of the valid angles, until the entire tissue is moved beyond the target plane.

We evaluate our method on a test set of 100 previously unseen configurations, varying in tissue size and surgical tool pose. A representative manipulation sequence produced by our method is shown in Figure 7. In addition, we compare *DefFusionNet* against the current state-of-the-art goal generation model, *DefGoalNet* [7], which is trained on the same 1000 demonstrations and evaluated under identical conditions.

We additionally investigate the impact of dataset size on the performance of *DefFusionNet* by training separate models using 10, 100, and 1000 demonstrations. The full set of 1000 demonstrations is generated by executing the robot demonstrator on 1000 distinct tissue samples, each with a randomly sampled surgical tool pose and retraction angle. The smaller subsets are uniformly sampled from this full set. We train the 10-demonstration model five times with five different random subsets to examine performance variance at small data scales.

Qualitatively, as illustrated in Figure 2, *DefFusionNet* captures much more effectively the underlying two-mode goal distribution, producing two distinct and equally valid goals. Both predicted goals are realistic and semantically coherent. In contrast, *DefGoalNet* average the two modes, resulting in a single unrealistic and physically impossible goal point cloud.

Quantitatively, we employ two evaluation metrics described

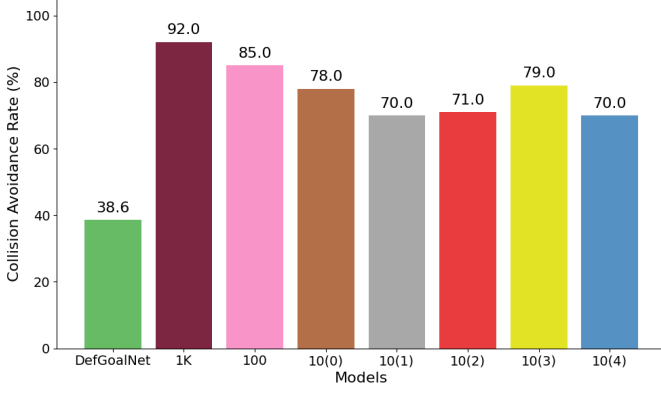


Fig. 8. **Simulated retraction results** - Collision avoidance rate metric (higher is better) on the test set across multiple training dataset sizes. From left to right: *DefGoalNet* [7] (current state-of-the-art), followed by *DefFusionNet* (ours) trained with 1000, 100, and 10 demonstrations. We train the 10-demonstration model 5 times with 5 different subsets of demonstrations, labeled 10(0-4).

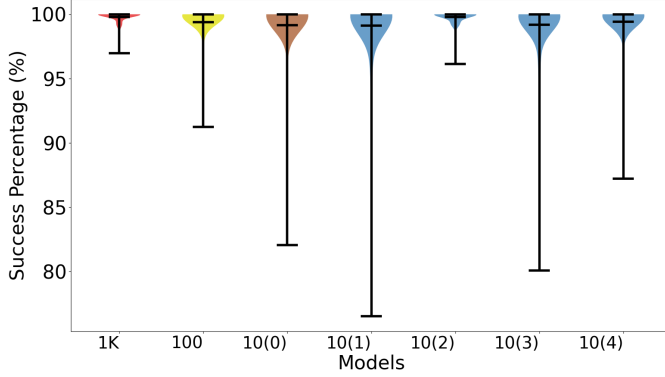


Fig. 9. **Simulated retraction results** - Success percentage metric.

below. The first and most important class of evaluation metrics directly correlates with the task success rate. For each test configuration, we execute the *DeformerNet* policy conditioned on the predicted goal generated by *DefFusionNet* and record the final object point cloud. We assess the quality of each retraction trajectory using two sub-metrics. First, we compute the collision avoidance success rate across 100 test scenarios using Isaac Gym’s soft-to-rigid contact detection. For the subset of scenarios that successfully avoid collisions, we further calculate the percentage of points in the final point cloud that successfully pass through the target plane, a metric we refer to as the *success percentage*. As shown in Fig. 8 and 9, even with as few as 10 demonstrations, our method can still achieve a median collision avoidance success rate of more than 70% and a median success percentage of more than 95%. Notably, *DefFusionNet* using only 10 demonstrations already outperforms *DefGoalNet* using the full dataset with 1000 demonstrations.

The second metric is the Chamfer distance [46] between the predicted and ground-truth goal point clouds. Since the dataset contains two possible goals per context, we compute two Chamfer distances between a single predicted goal and each of the two ground-truth goals, and report the minimum. The Chamfer results and example predicted goals are visualized in Fig. 10. Unsurprisingly, as we increase the demonstration dataset size, the generated goals become more realistic and

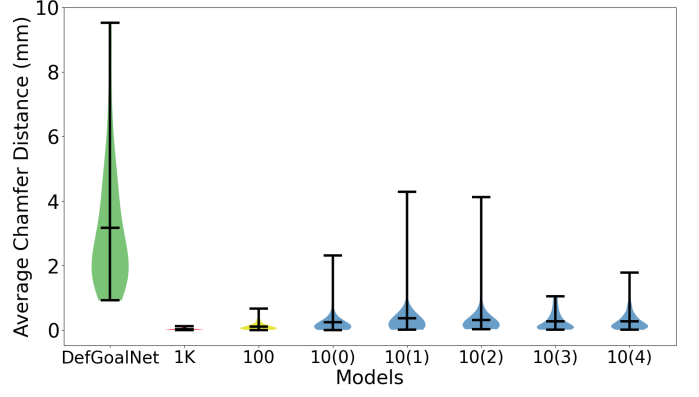


Fig. 10. **Simulated retraction results** - Chamfer distance metric (lower is better) between predicted and ground truth goal point clouds, on the test set, across multiple training dataset sizes.

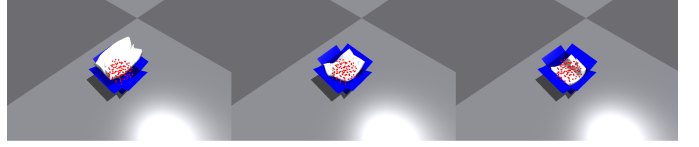


Fig. 11. Sample manipulation sequence on the object packaging task, in simulation. The red point cloud visualizes the goal shape generated by *DefFusionNet*.

easily interpretable; however, the structure of the goal is visible even from 10 demonstrations.

2) *Manufacturing-inspired Object Packaging*: To demonstrate that *DefFusionNet* not only performs in surgical tasks but also excels in industrial applications, we conduct additional experiments on an object packaging task (Fig. 11) inspired by a common warehouse scenario: the item to be packaged is initially larger than the container’s opening. This task requires the coordinated effort of two robotic arms to safely deform and compress the object, allowing it to pass through the opening without causing damage. The contextual point cloud $\mathcal{P}_T$ for this task corresponds to a partial-view point cloud of the box container.

Demonstrations are collected using a scripted multimodal robot policy, as illustrated in Figure 12. The procedure begins with the robot grasping the deformable object and positioning it such that its centroid aligns with that of the container, excluding the z-component (which is aligned with gravity). Next, a random rotation angle is sampled, and the object is allowed to freely rotate around the z-axis according to this angle. Finally, the robot lowers the object along the z-axis while simultaneously bringing the two end-effectors closer together, effectively folding the object to fit through the container opening. In contrast to the previous surgical experiment,

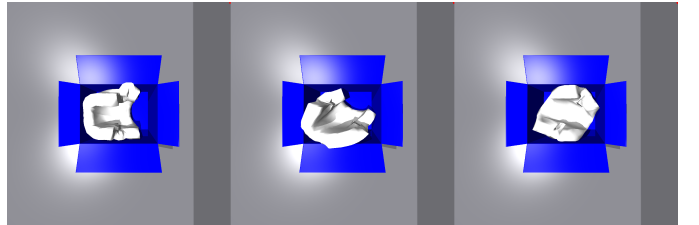


Fig. 12. Example demonstrations for the object packaging task. For a given container, the deformable object can be arranged in multiple valid configurations.

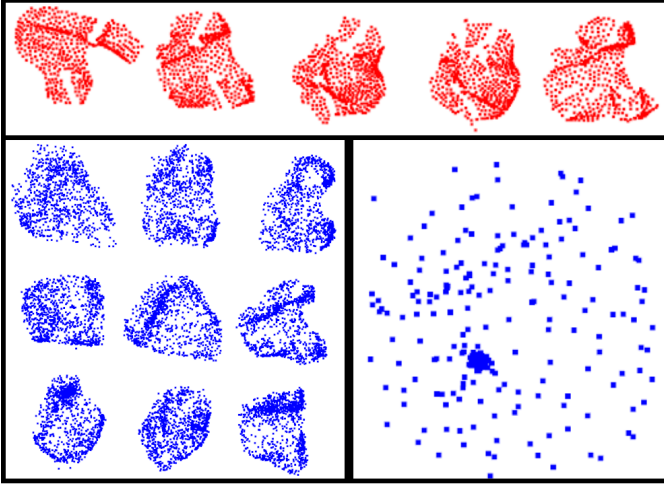


Fig. 13. **(Red)** Ground-truth goal point clouds collected from expert demonstrator. **(Blue)** Predicted goals from *DefFusionNet* (bottom left) compared to *DefGoalNet* (bottom right). *DefFusionNet* generates diverse, multimodal goal shapes, while *DefGoalNet*'s prediction is physically infeasible.

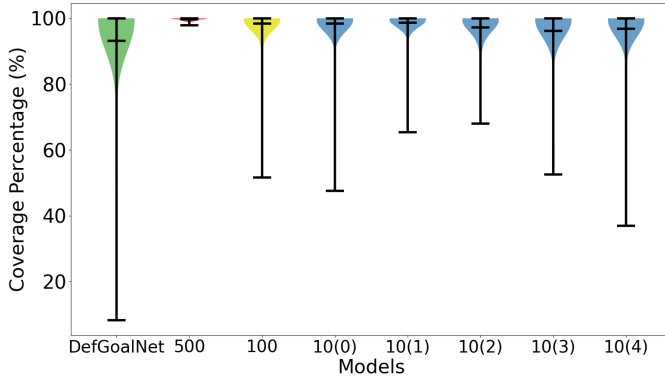


Fig. 14. **Simulated object packaging** - Coverage percentage metric (higher is better). From left to right: *DefGoalNet* [7] (current state-of-the-art), followed by *DefFusionNet* (ours) trained with 500, 100, and 10 demonstrations.

we do not enforce a two-mode goal distribution for the object packaging task. Instead, the rotation angle is sampled from a continuous uniform distribution over $[-\pi, \pi]$, resulting in a complex continuous goal distribution with one high-variance mode. For clearer evaluation, we collect five distinct goal shapes for each tissue size and context, corresponding to different rotation outcomes. However, this does not imply a five-mode goal distribution, as the rotation angles are still sampled from a continuous distribution.

Similar to the surgical retraction experiment, we train *DefFusionNet* on a varying number of demonstrations: 10, 100, and 1000. A representative manipulation sequence is visualized in Fig. 11.

We compute the *coverage percentage* to quantify the task success, which measures the percentage of object volume being contained by container. To evaluate, we first randomize the deformable object size and container pose for each test scenario. We then execute the *DeformerNet* policy conditioned on the predicted goal generated by *DefFusionNet* and record the final object point cloud, then compute percentage of points successfully contained. We visualize the results qualitatively in Fig. 13 and quantitatively in Fig. 14. At a dataset size of 100, our method starts achieving competitive performance with a

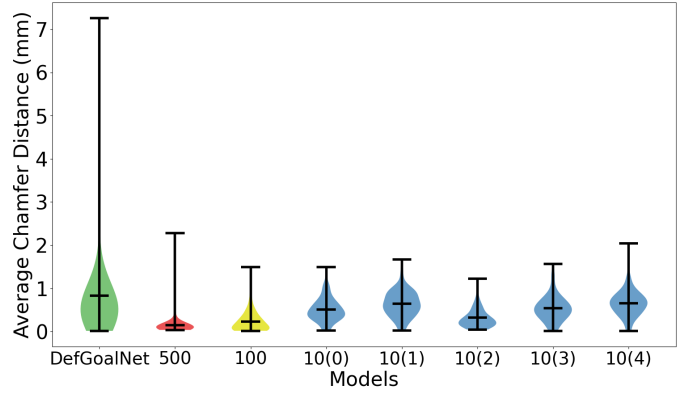


Fig. 15. **Simulated object packaging** - Chamfer distance between predicted and ground truth goal point clouds, on the test set.

median coverage percentage of almost 100%. *DefFusionNet* again captures effectively the multimodal goal distribution, while *DefGoalNet* still produces physically impractical goal point clouds. *DefFusionNet* using only 10 demonstrations also outperforms *DefGoalNet* using the full dataset with 1000 demonstrations.

Fig. 15 visualizes the Chamfer distance between the predicted and ground-truth goal point clouds on a test set of 100 unseen demonstrations. Similar to the surgical retraction task, *DefFusionNet* significantly outperforms *DefGoalNet*. Also, as the number of demonstrations increases, the predicted goals more closely resemble the ground-truth shapes.

VI. CONCLUSION

We have presented a pipeline for learning deformable object manipulation from demonstrations. At the heart of this pipeline is *DefFusionNet*, a neural network that learns a multimodal distribution over diverse goal shapes capable of successfully completing the given task. We evaluate *DefFusionNet* across a broad range of robotic tasks spanning both surgery and manufacturing applications. Our experiments show that *DefFusionNet* consistently outperforms the current state-of-the-art goal prediction method—which models a single deterministic goal rather than a distribution as in our approach—across multiple metrics and task domains, while also requiring less training data. Notably, our approach enables effective task-specific goal generation from only a limited number of demonstrations, while still benefiting from a generic control policy trained on a large and diverse dataset that is cheap to acquire and agnostic to the specific downstream task.

REFERENCES

- [1] M. T. Mason, "Toward Robotic Manipulation," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 1, no. 1, pp. 1–28, 2018.
- [2] J. Sanchez, J. A. C. Ramon, B.-C. Bouzgarrou, and Y. Mezouar, "Robotic Manipulation and Sensing of Deformable Objects in Domestic and Industrial Applications: A Survey," *Intl. Journal of Robotics Research*, vol. 37, no. 7, pp. 688–716, 2018.
- [3] I. Huang, Y. Narang, C. Eppner, B. Sundaralingam, M. Macklin, T. Hermans, and D. Fox, "DefGraspSim: Simulation-based grasping of 3D deformable objects," in *RSS Workshop on Deformable Object Simulation in Robotics (DO-Sim)*, 2021.
- [4] J. Zhu, A. Cherubini, C. Dune, D. Navarro-Alarcon, F. Alambeigi, D. Berenson, F. Ficuciello, K. Harada, J. Kober, X. Li, *et al.*, "Challenges and outlook in robotic manipulation of deformable objects," *IEEE Robotics & Automation Magazine*, vol. 29, no. 3, pp. 67–77, 2022.

- [5] Y. Wu, W. Yan, T. Kurutach, L. Pinto, and P. Abbeel, "Learning to manipulate deformable objects without demonstrations," *Robotics: Science and Systems*, 2020.
- [6] B. Thach, B. Y. Cho, T. Hermans, and A. Kuntz, "Deformernet: Learning bimanual manipulation of 3d deformable objects," *ArXiv*, 2023.
- [7] B. Thach, T. Watts, S.-H. Ho, T. Hermans, and A. Kuntz, "Defgoalnet: Contextual goal learning from demonstrations for deformable object manipulation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 3145–3152.
- [8] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [9] Q. Lu, M. Van der Merwe, B. Sundaralingam, and T. Hermans, "Multi-fingered grasp planning via inference in deep neural networks: Outperforming sampling by learning differentiable models," *IEEE Robotics & Automation Magazine*, vol. 27, no. 2, pp. 55–65, 2020.
- [10] A. Mousavian, C. Eppner, and D. Fox, "6-dof grasnet: Variational grasp generation for object manipulation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 2901–2910.
- [11] A. Murali, A. Mousavian, C. Eppner, C. Paxton, and D. Fox, "6-dof grasping for target-driven object manipulation in clutter," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 6232–6238.
- [12] X. Deng, Y. Xiang, A. Mousavian, C. Eppner, T. Bretl, and D. Fox, "Self-supervised 6D Object Pose Estimation for Robot Manipulation," *IEEE Intl. Conf. on Robotics and Automation*, pp. 3665–3671, 2020.
- [13] Q. Lu, M. Van der Merwe, and T. Hermans, "Multi-fingered active grasp learning," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 8415–8422.
- [14] M. Van der Merwe, Q. Lu, B. Sundaralingam, M. Matak, and T. Hermans, "Learning continuous 3d reconstructions for geometrically aware grasping," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 11 516–11 522.
- [15] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, *et al.*, "Rt-1: Robotics transformer for real-world control at scale," *arXiv preprint arXiv:2212.06817*, 2022.
- [16] D. Navarro-Alarcon, Y. Liu, J. G. Romero, and P. Li, "Visually servoed deformation control by robot manipulators," in *2013 IEEE International Conference on Robotics and Automation*. IEEE, 2013, pp. 5259–5264.
- [17] F. Alamebeigi, Z. Wang, R. Hegeman, Y.-H. Liu, and M. Armand, "Autonomous data-driven manipulation of unknown anisotropic deformable tissues using unmodelled continuum manipulators," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 254–261, 2018.
- [18] D. Navarro-Alarcon, H. M. Yip, Z. Wang, Y.-H. Liu, F. Zhong, T. Zhang, and P. Li, "Automatic 3-d manipulation of soft objects by robotic arms with an adaptive deformation model," *IEEE Transactions on Robotics*, vol. 32, no. 2, pp. 429–441, 2016.
- [19] J. Qi, D. Li, Y. Gao, P. Zhou, and D. Navarro-Alarcon, "Model predictive manipulation of compliant objects with multi-objective optimizer and adversarial network for occlusion compensation," *arXiv preprint arXiv:2205.09987*, 2022.
- [20] F. Alamebeigi, Z. Wang, R. Hegeman, Y.-H. Liu, and M. Armand, "A robust data-driven approach for online learning and manipulation of unmodeled 3-d heterogeneous compliant objects," *IEEE Robotics and Automation Letters*, vol. 3, no. 4, pp. 4140–4147, 2018.
- [21] M. Shetab-Bushehri, M. Aranda, Y. Mezouar, and E. Ozgur, "Lattice-based shape tracking and servoing of elastic objects," *arXiv preprint arXiv:2209.01832*, 2022.
- [22] Z. Hu, T. Han, P. Sun, J. Pan, and D. Manocha, "3-D Deformable Object Manipulation Using Deep Neural Networks," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4255–4261, 2019.
- [23] R. B. Rusu, G. Bradski, R. Thibaux, and J. Hsu, "Fast 3D recognition and pose using the Viewpoint Feature Histogram," *IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems*, pp. 2155–2162, 2010.
- [24] B. Thach, B. Y. Cho, A. Kuntz, and T. Hermans, "Learning visual shape control of novel 3d deformable objects from partial-view point clouds," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 8274–8281.
- [25] W. Wu, Z. Qi, and L. Fuxin, "Pointconv: Deep convolutional networks on 3d point clouds," *ArXiv*, 2018.
- [26] D. W. Shu, S. W. Park, and J. Kwon, "3d point cloud generative adversarial network based on tree structured graph convolutions," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 3859–3868.
- [27] P. Achlioptas, O. Diamanti, I. Mitliagkas, and L. Guibas, "Learning representations and generative models for 3d point clouds," in *International conference on machine learning*. PMLR, 2018, pp. 40–49.
- [28] M. S. Arshad and W. J. Beksi, "A progressive conditional generative adversarial network for generating dense and colored 3d point clouds," in *2020 International Conference on 3D Vision (3DV)*. IEEE, 2020, pp. 712–722.
- [29] A. Pumarola, S. Popov, F. Moreno-Noguer, and V. Ferrari, "C-flow: Conditional generative flow models for images and 3d point clouds," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 7949–7958.
- [30] S. van Waveren, C. Pek, I. Leite, J. Tumova, and D. Kragic, "Large-scale scenario generation for robotic manipulation via conditioned generative models," 2022.
- [31] Y. Du, S. Yang, B. Dai, H. Dai, O. Nachum, J. Tenenbaum, D. Schuurmans, and P. Abbeel, "Learning universal policies via text-guided video generation," *Advances in neural information processing systems*, vol. 36, pp. 9156–9172, 2023.
- [32] O. Mees, L. Hermann, and W. Burgard, "What matters in language conditioned robotic imitation learning over unstructured data," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 205–11 212, 2022.
- [33] H. Zhou, X. Yao, O. Mees, Y. Meng, T. Xiao, Y. Bisk, J. Oh, E. Johns, M. Shridhar, D. Shah, *et al.*, "Bridging language and action: A survey of language-conditioned robot manipulation," *arXiv e-prints*, pp. arXiv–2312, 2023.
- [34] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, p. 02783649241273668, 2023.
- [35] J. Pavlasek, J. J. Z. Mah, R. Xu, O. C. Jenkins, and F. Ramos, "Stein variational belief propagation for multi-robot coordination," *IEEE Robotics and Automation Letters*, 2024.
- [36] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.
- [37] U. Singer, A. Polyak, T. Hayes, X. Yin, J. An, S. Zhang, Q. Hu, H. Yang, O. Ashual, O. Gafni, *et al.*, "Make-a-video: Text-to-video generation without text-video data," *arXiv preprint arXiv:2209.14792*, 2022.
- [38] Q. Huang, D. S. Park, T. Wang, T. I. Denk, A. Ly, N. Chen, Z. Zhang, Z. Zhang, J. Yu, C. Frank, *et al.*, "Noise2music: Text-conditioned music generation with diffusion models," *arXiv preprint arXiv:2302.03917*, 2023.
- [39] S. Luo and W. Hu, "Diffusion probabilistic models for 3d point cloud generation," pp. 2837–2845, 2021.
- [40] D. P. Kingma, "Auto-encoding variational bayes," *arXiv preprint arXiv:1312.6114*, 2013.
- [41] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "Pointnet: Deep learning on point sets for 3d classification and segmentation," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 652–660, 2017.
- [42] S. Hochreiter, "Long short-term memory," *Neural Computation MIT-Press*, 1997.
- [43] P. Kazanzides, Z. Chen, A. Deguet, G. S. Fischer, R. H. Taylor, and S. P. DiMaio, "An open-source research kit for the da Vinci® Surgical System," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, May 2014, pp. 6434–6439.
- [44] J. Liang, V. Makovychuk, A. Handa, N. Chentanez, M. Macklin, and D. Fox, "GPU-Accelerated Robotic Simulation for Distributed Reinforcement Learning," *arXiv:1810.05762*, 2018.
- [45] A. Ernst, D. Feller-Kopman, H. D. Becker, and A. C. Mehta, "Central airway obstruction," *American journal of respiratory and critical care medicine*, vol. 169, no. 12, pp. 1278–1297, 2004.
- [46] H. Fan, H. Su, and L. J. Guibas, "A point set generation network for 3d object reconstruction from a single image," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 605–613.